Lab 3 Reflection — Composite Pattern

Riejed Aniko L. Macatula | BSIT 3A | Integrative Programming and Technologies

The smell

The without-composite version represents a forum thread with PostNaive and ThreadNaive, two unrelated classes with no shared type. renderNaive() is the only place that knows how to walk the tree, and it does that with if ($el instanceof PostNaive) followed by if ($el instanceof ThreadNaive), recursing into the second branch and returning an empty string if neither matches. That function has to change every time a new node type shows up. Adding a Poll type means opening renderNaive(), adding a third instanceof check, and trusting that every other place in the codebase that walks this tree gets the same treatment. The tree itself is fine; the traversal logic is fragile because it's centralized in one function instead of belonging to each node.

The fix

Composite gives leaves and branches the same interface, ForumComponent, with one method: display(depth). Post is the leaf and just prints itself. Thread is the composite; it holds a children array typed as ForumComponent[] and, once I filled in the loop, calls $child->display($depth + 1) on each one. Neither class needs to know what type the other is. Adding a new node type means writing a class that implements ForumComponent, nothing else. I tested that by adding Bundle, a leaf representing a pre-set welcome pack, and dropping it into an existing Thread without touching Thread, Post, or the client code that calls display().

The RAG side of the lab is the same shape wearing different names. TextComponent replaces ForumComponent, Chunk is the leaf, and Section/Document are composites holding TextComponent[] children. getText() recursively concatenates a node's own text with its children's text, and embed() recurses down to each Chunk the same way display() recurses down to each Post. Building this variant right after the forum one made it clear that Composite isn't really about forums or documents; it's about any part-whole tree where the client wants to treat a leaf and a branch the same way.

Gamma citation

Gamma et al. (1994, p. 163) describe Composite as a pattern to "compose objects into tree structures to represent part-whole hierarchies" so that "clients can treat individual objects and compositions of objects uniformly." That uniform treatment is exactly what replaced the instanceof chain: Thread::display() and Document::getText() call the same method on every child regardless of whether that child is a leaf or another composite.